LAB EXPERIMENT – 1

Examination Result and Grace-Mark Validation

Problem Statement: Process marks in three subjects, apply subject-wise pass criteria, use available grace marks where possible, and print the final result.

Aim: To check whether a student passes by using a limited number of grace marks.

Input – Process – Output: Input: marks in three subjects (m1, m2, m3), pass mark, and available grace marks. Process: calculate each subject's deficit and use grace marks in order. Output: updated marks and PASS or FAIL.

1. Algorithm

1. Start.

2. Read m1, m2, m3, pass_mark, and grace.

3. Set status = PASS.

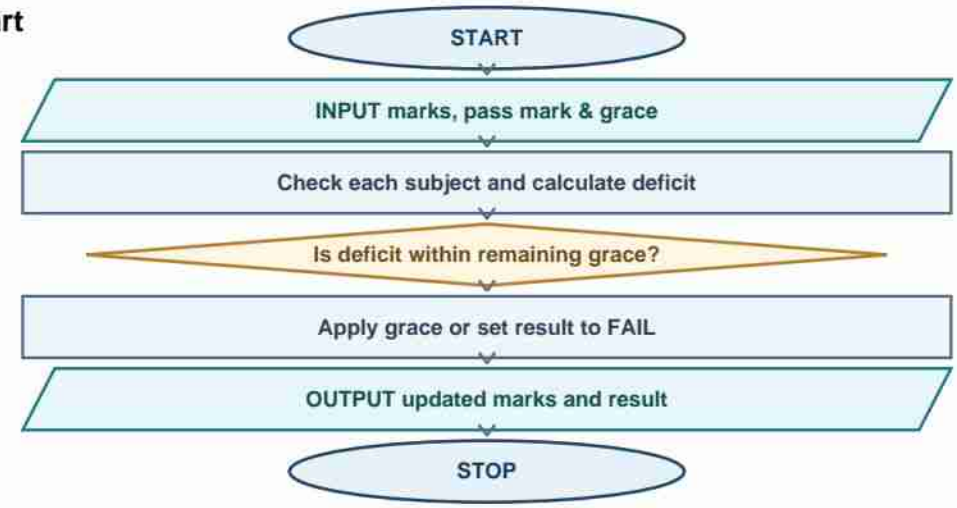
4. For each subject mark: if it is below pass_mark, calculate diff = pass_mark - mark.

5. If diff is not greater than the remaining grace, add diff to the mark and subtract diff from grace; otherwise set status = FAIL.

6. Display the updated marks and result.

7. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT m1, m2, m3, pass_mark, grace
  status = "PASS"

  FOR each mark in [m1, m2, m3]
      IF mark < pass_mark
          diff = pass_mark - mark
          IF diff <= grace
              mark = mark + diff
              grace = grace - diff
          ELSE
              status = "FAIL"
          END IF
      END IF
  END FOR

  PRINT updated marks and status
END
```

LAB EXPERIMENT – 1 (CONTINUED)

4. Python Program

```
m1 = int(input("Enter marks in subject 1: "))
m2 = int(input("Enter marks in subject 2: "))
m3 = int(input("Enter marks in subject 3: "))
pass_mark = int(input("Enter pass mark: "))
grace = int(input("Enter grace marks: "))

status = "PASS"

if m1 < pass_mark:
    diff = pass_mark - m1
    if diff <= grace:
        m1 = m1 + diff
        grace = grace - diff
    else:
        status = "FAIL"

if m2 < pass_mark:
    diff = pass_mark - m2
    if diff <= grace:
        m2 = m2 + diff
        grace = grace - diff
    else:
        status = "FAIL"

if m3 < pass_mark:
    diff = pass_mark - m3
    if diff <= grace:
        m3 = m3 + diff
        grace = grace - diff
    else:
        status = "FAIL"

print("Updated Marks:", m1, m2, m3)
print("Result:", status)
```

5. Sample Output

```
Enter marks in subject 1: 33
Enter marks in subject 2: 40
Enter marks in subject 3: 50
Enter pass mark: 35
Enter grace marks: 5
Updated Marks: 35 40 50
Result: PASS
```

6. Test Cases

m1=33, m2=40, m3=50, pass=35, grace=5	Updated Marks: 35 40 50; Result: PASS
m1=25, m2=60, m3=70, pass=35, grace=5	Updated Marks: 25 60 70; Result: FAIL

Note: The supplied case study does not give a separate maximum-mark input, so this solution follows its stated grace-mark calculation and does not add a maximum-mark cap.

LAB EXPERIMENT – 2

Attendance Recovery Computation

Problem Statement: Calculate the current attendance percentage and find the minimum number of consecutive classes a student must attend to reach a target percentage.

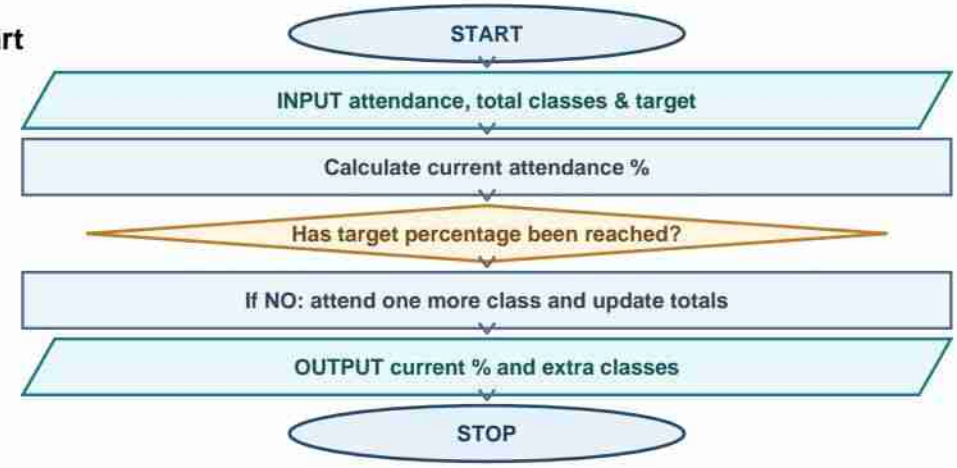
Aim: To calculate current attendance and the additional classes required to reach the target.

Input – Process – Output: Input: attended classes, total conducted classes, and target percentage. Process: calculate the current percentage and use a while loop to simulate attending more classes. Output: current percentage and number of additional classes needed.

1. Algorithm

1. Start.
2. Read attended, total_classes, and target.
3. Calculate current_pct = attended / total_classes × 100.
4. Copy the values into temp_att and temp_tot. Set needed = 0.
5. While the simulated attendance percentage is below target, increase temp_att, temp_tot, and needed by 1.
6. Display current_pct and needed.
7. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT attended, total_classes, target
  current_pct = (attended / total_classes) * 100
  temp_att = attended
  temp_tot = total_classes
  needed = 0

  WHILE (temp_att / temp_tot) * 100 < target
      temp_att = temp_att + 1
      temp_tot = temp_tot + 1
      needed = needed + 1
  END WHILE

  PRINT current_pct
  PRINT needed
END
```

4. Python Program

```
attended = int(input("Enter attended classes: "))
total_classes = int(input("Enter total classes: "))
target = float(input("Enter target percentage: "))

current_pct = (attended / total_classes) * 100
temp_att = attended
temp_tot = total_classes
needed = 0

while (temp_att / temp_tot) * 100 < target:
    temp_att = temp_att + 1
    temp_tot = temp_tot + 1
    needed = needed + 1

print("Current Attendance:", current_pct)
print("Classes needed to reach target:", needed)
```

5. Sample Output

```
Enter attended classes: 28
Enter total classes: 40
Enter target percentage: 75
Current Attendance: 70.0
Classes needed to reach target: 8
```

6. Test Cases

attended=28, total=40, target=75	Current attendance: 70.0%; Classes needed: 8
attended=30, total=40, target=75	Current attendance: 75.0%; Classes needed: 0

Note: Enter a positive total_classes value. Division by zero is not valid.

LAB EXPERIMENT – 3

ATM Transaction Sequence Simulation

Problem Statement: Simulate deposits and withdrawals, maintain the account balance, enforce a daily withdrawal limit, reject invalid withdrawals, and show a transaction summary.

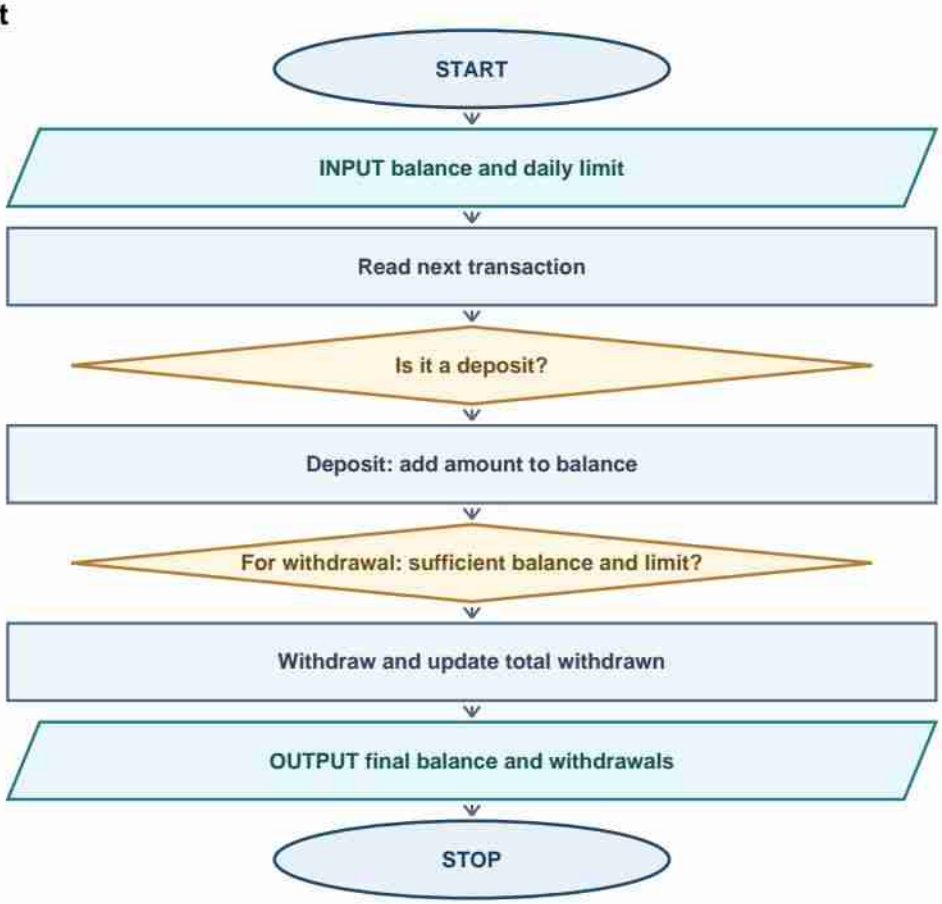
Aim: To process a sequence of deposits and withdrawals using conditions and loops.

Input – Process – Output: Input: opening balance and daily withdrawal limit. The sample transaction list is [1000, -2000, -4000, 500], where positive values are deposits and negative values are withdrawals. Output: transaction messages, final balance, and total withdrawn.

1. Algorithm

1. Start.
2. Read the opening balance and daily withdrawal limit.
3. Set total_withdrawn = 0 and use the transaction list.
4. For each transaction: if it is positive, add it to the balance.
5. Otherwise, convert it to a positive withdrawal amount.
6. If the amount is greater than the balance, display insufficient balance.
7. Else if total_withdrawn + amount is greater than the daily limit, display limit exceeded.
8. Otherwise subtract the amount from the balance and add it to total_withdrawn.
9. Display final balance and total withdrawn. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT balance, daily_limit
  total_withdrawn = 0
  transactions = [1000, -2000, -4000, 500]

  FOR each t in transactions
      IF t > 0
          balance = balance + t
          PRINT deposit message
      ELSE
          amount = -t
          IF amount > balance
              PRINT "Insufficient balance"
          ELSE IF total_withdrawn + amount > daily_limit
              PRINT "Exceeds daily limit"
          ELSE
              balance = balance - amount
              total_withdrawn = total_withdrawn + amount
              PRINT withdrawal message
          END IF
      END IF
  END FOR

  PRINT balance and total_withdrawn
END
```

4. Python Program

```
balance = int(input("Enter initial balance: "))
daily_limit = int(input("Enter daily withdrawal limit: "))

total_withdrawn = 0
transactions = [1000, -2000, -4000, 500]

for t in transactions:
    if t > 0:
        balance = balance + t
        print("Deposited:", t, "New Balance:", balance)
    else:
        amount = -t

        if amount > balance:
            print("Failed: Insufficient balance for", amount)
        elif total_withdrawn + amount > daily_limit:
            print("Failed: Exceeds daily limit for", amount)
        else:
            balance = balance - amount
            total_withdrawn = total_withdrawn + amount
            print("Withdrew:", amount, "New Balance:", balance)

print("Final Balance:", balance)
print("Total Withdrawn:", total_withdrawn)
```

5. Sample Output

```
Enter initial balance: 5000
Enter daily withdrawal limit: 10000
Deposited: 1000 New Balance: 6000
Withdrew: 2000 New Balance: 4000
Failed: Insufficient balance for 4000
Deposited: 500 New Balance: 4500
Final Balance: 4500
Total Withdrawn: 2000
```

6. Test Cases

balance=5000, limit=10000, transactions=[1000, -2000]	Final Balance: 4000; Total Withdrawn: 2000
balance=2000, limit=5000, transactions=[-3000]	Insufficient balance; Final Balance: 2000

Note: The manual's sample program uses a fixed transaction list. This beginner-level version keeps that list fixed so the focus remains on if-else conditions and for loops.

LAB EXPERIMENT – 4

Advanced Numeric Pattern Generator (Hollow Diamond)

Problem Statement: Print a symmetric hollow diamond number pattern based on the number of rows entered by the user, using nested loops.

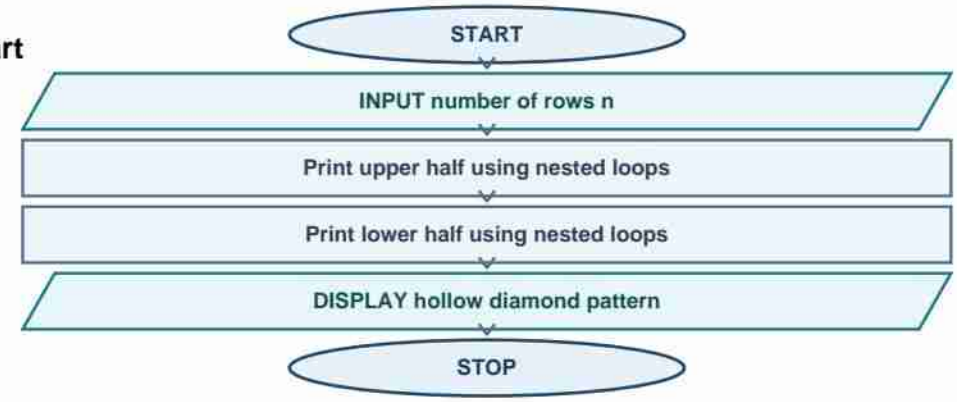
Aim: To create a hollow diamond pattern using nested for loops.

Input – Process – Output: Input: n, the number of rows in the upper half. Process: print leading spaces and boundary numbers in the upper and lower halves. Output: a hollow diamond pattern.

1. Algorithm

1. Start and read n.
2. For i from 1 to n, print n-i leading spaces.
3. For j from 1 to 2×i-1, print i at the first and last positions; otherwise print a space.
4. Move to the next line.
5. Repeat the same process for i from n-1 down to 1.
6. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT n
  FOR i FROM 1 TO n
      PRINT (n-i) leading spaces
      FOR j FROM 1 TO (2*i-1)
          IF j == 1 OR j == (2*i-1)
              PRINT i without newline
          ELSE
              PRINT space without newline
          END IF
      END FOR
      PRINT newline
  END FOR

  FOR i FROM n-1 DOWN TO 1
      Repeat the same space and boundary-number steps
  END FOR
END
```

4. Python Program

```
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    for space in range(n - i):
        print(" ", end="")

    for j in range(1, 2 * i):
        if j == 1 or j == (2 * i - 1):
            print(i, end="")
        else:
            print(" ", end="")
    print()

for i in range(n - 1, 0, -1):
    for space in range(n - i):
        print(" ", end="")

    for j in range(1, 2 * i):
        if j == 1 or j == (2 * i - 1):
            print(i, end="")
        else:
            print(" ", end="")
    print()
```

5. Sample Output

```
Enter number of rows: 3
  1
 2 2
3   3
 2 2
  1
```

6. Test Cases

n=3	A symmetric 5-line hollow diamond is printed, with 1, 2, and 3 on its boundary.
n=4	A symmetric 7-line hollow diamond is printed, with 1, 2, 3, and 4 on its boundary.

Note: The pattern uses print(..., end="") to keep characters on the same line. print() moves to the next line.

LAB EXPERIMENT – 5

Password Policy Validation Engine

Problem Statement: Check a sample password for minimum length and the presence of uppercase letters, lowercase letters, and digits.

Aim: To validate a password using basic string methods and Boolean variables.

Input – Process – Output: Input: a password string. Process: check whether its length is at least 8 and scan its characters using isupper(), islower(), and isdigit(). Output: validation messages.

1. Algorithm

1. Start and read the password.

2. Set has_upper, has_lower, and has_digit to False.

3. If the password length is less than 8, print the length failure message.

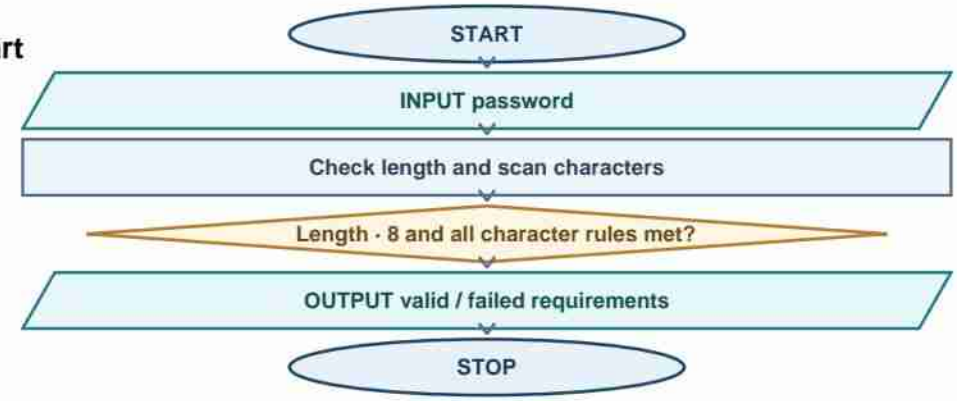
4. Visit each character in the password.

5. If the character is uppercase, set has_upper to True; if lowercase, set has_lower to True; if a digit, set has_digit to True.

6. If the length and all three flags are valid, print Password is valid. Otherwise print the relevant failure messages.

7. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT password
  has_upper = False
  has_lower = False
  has_digit = False

  FOR each ch in password
      IF ch is uppercase: has_upper = True
      IF ch is lowercase: has_lower = True
      IF ch is a digit: has_digit = True
  END FOR

  IF length(password) >= 8 AND has_upper AND has_lower AND has_digit
      PRINT "Password is valid"
  ELSE
      PRINT the failed requirements
  END IF
END
```

4. Python Program

```
password = input("Enter password: ")

has_upper = False
has_lower = False
has_digit = False

for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True

if len(password) < 8:
    print("Failed: Password must be at least 8 characters long")
if not has_upper:
    print("Failed: Missing uppercase letter")
if not has_lower:
    print("Failed: Missing lowercase letter")
if not has_digit:
    print("Failed: Missing numeric digit")

if len(password) >= 8 and has_upper and has_lower and has_digit:
    print("Password is valid")
```

5. Sample Output

```
Enter password: Pass1234
Password is valid
```

6. Test Cases

password="Pass1234"	Password is valid
password="pass"	Failed: Password must be at least 8 characters long; Missing uppercase letter; Missing numeric digit

Note: This is a classroom exercise about string checks. It is not a complete real-world password security system.

LAB EXPERIMENT – 6

String-Based Examination Answer Evaluation

Problem Statement: Compare a student's answer string with an answer key, count correct, incorrect, and unanswered responses, and calculate a score with negative marking.

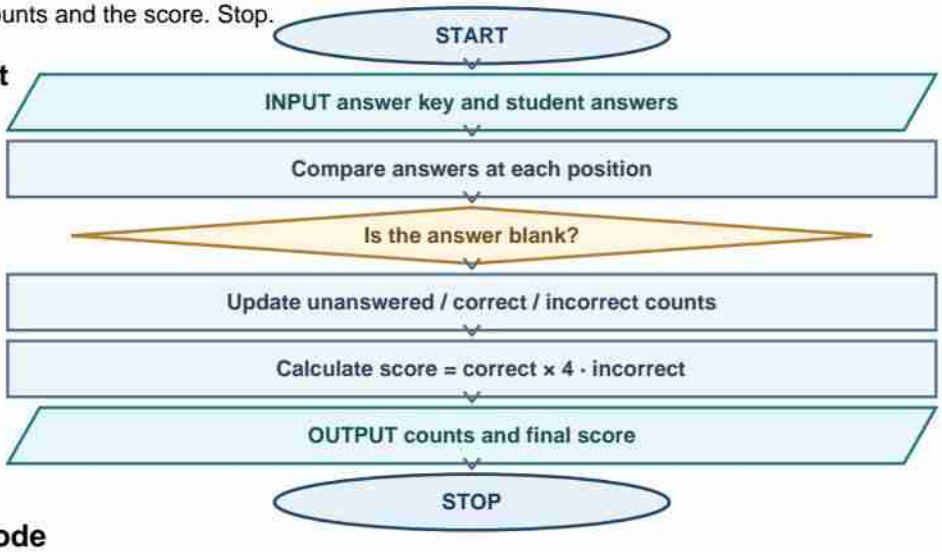
Aim: To evaluate answers using string indexing and a loop.

Input – Process – Output: Input: answer key and student answers. A space represents an unanswered question. Process: compare characters at matching positions; award +4 for correct, -1 for incorrect, and 0 for unanswered. Output: counts and final score.

1. Algorithm

1. Start and read the answer key and student answer string.
2. Set correct, incorrect, and unanswered to 0.
3. For every index in the answer key, check the student's answer at the same index.
4. If it is a space, increase unanswered; if it matches the key, increase correct; otherwise increase incorrect.
5. Calculate score = correct×4 - incorrect.
6. Display all counts and the score. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT key, answers
  correct = 0
  incorrect = 0
  unanswered = 0

  FOR i FROM 0 TO length(key)-1
      IF answers[i] == space
          unanswered = unanswered + 1
      ELSE IF answers[i] == key[i]
          correct = correct + 1
      ELSE
          incorrect = incorrect + 1
      END IF
  END FOR

  score = (correct * 4) - incorrect
  PRINT correct, incorrect, unanswered, score
END
```

4. Python Program

```
key = input("Enter answer key: ")
answers = input("Enter student answers: ")

correct = 0
incorrect = 0
unanswered = 0

for i in range(len(key)):
    if answers[i] == " ":
        unanswered = unanswered + 1
    elif answers[i] == key[i]:
        correct = correct + 1
    else:
        incorrect = incorrect + 1

score = (correct * 4) - incorrect

print("Correct:", correct)
print("Incorrect:", incorrect)
print("Unanswered:", unanswered)
print("Final Score:", score)
```

5. Sample Output

```
Enter answer key: ABCDABCD
Enter student answers: AB CDABC
Correct: 6
Incorrect: 1
Unanswered: 1
Final Score: 23
```

6. Test Cases

key="ABCDABCD", answers="AB CDABC"	Correct: 6; Incorrect: 1; Unanswered: 1; Score: 23
key="AAAA", answers="AAAA"	Correct: 4; Incorrect: 0; Unanswered: 0; Score: 16

Note: The two strings must have equal length. Each character position represents one question.

LAB EXPERIMENT – 7

Text Compression Using Character Runs

Problem Statement: Encode a string by grouping consecutive repeated characters, then compare the original string length with the encoded string length.

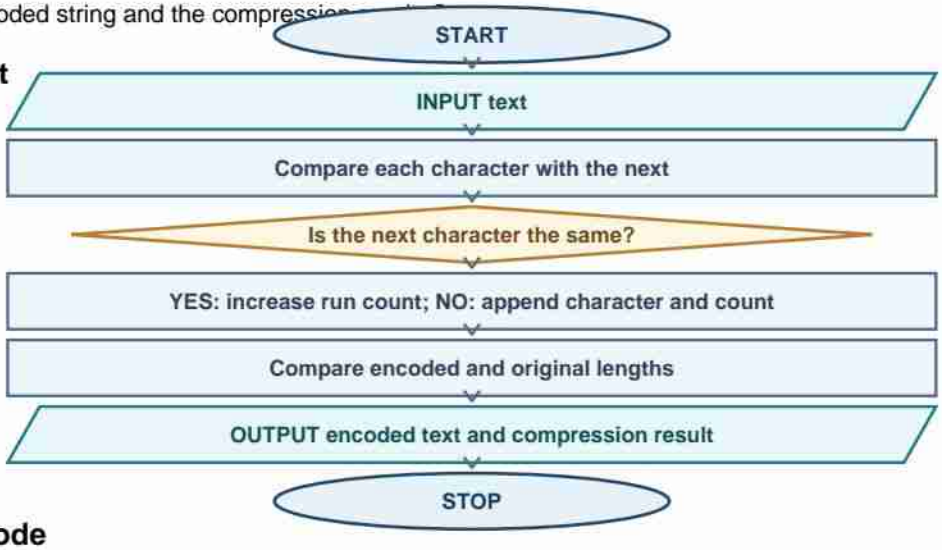
Aim: To perform simple run-length encoding using a for loop and string concatenation.

Input – Process – Output: Input: a text string. Process: count consecutive repeated characters and append each character followed by its count. Output: encoded text and whether compression reduces the length.

1. Algorithm

1. Start and read text.
2. Set encoded to an empty string and count to 1.
3. Visit each character by its index.
4. If the next character exists and is the same, increase count.
5. Otherwise append the current character and count to encoded, then reset count to 1.
6. Compare the lengths of encoded and original text.
7. Print the encoded string and the compression

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT text
  encoded = ""
  count = 1

  FOR i FROM 0 TO length(text)-1
      IF i+1 < length(text) AND text[i] == text[i+1]
          count = count + 1
      ELSE
          encoded = encoded + text[i] + STRING(count)
          count = 1
      END IF
  END FOR

  PRINT encoded
  IF length(encoded) < length(text)
      PRINT "Compression is beneficial"
  ELSE
      PRINT "Compression is not beneficial"
  END IF
END
```

4. Python Program

```
text = input("Enter text: ")

encoded = ""
count = 1

for i in range(len(text)):
    if i + 1 < len(text) and text[i] == text[i + 1]:
        count = count + 1
    else:
        encoded = encoded + text[i] + str(count)
        count = 1

print("Original Text:", text)
print("Encoded Text:", encoded)

if len(encoded) < len(text):
    print("Compression is beneficial")
else:
    print("Compression is not beneficial")
```

5. Sample Output

```
Enter text: aaabbccccd
Original Text: aaabbccccd
Encoded Text: a3b2c4d1
Compression is beneficial
```

6. Test Cases

text="aaabbccccd"	Encoded: a3b2c4d1; Compression is beneficial
text="abcd"	Encoded: a1b1c1d1; Compression is not beneficial

Note: This simple encoding is for learning. It does not include a decoding method or special handling for digits already present in the input.

LAB EXPERIMENT – 8

Student Marks Matrix Analysis

Problem Statement: Analyze a two-dimensional marks matrix, calculate each student's average and each subject's average, and find the top-performing student.

Aim: To process a nested list using nested loops and find the highest student average.

Input – Process – Output: Input: a nested list in which each row contains one student's marks and each column represents a subject. Process: calculate row-wise and column-wise averages and find the largest student average. Output: student averages, subject averages, and top student index.

1. Algorithm

1. Start and initialize the marks matrix.

2. Find the number of students and subjects.

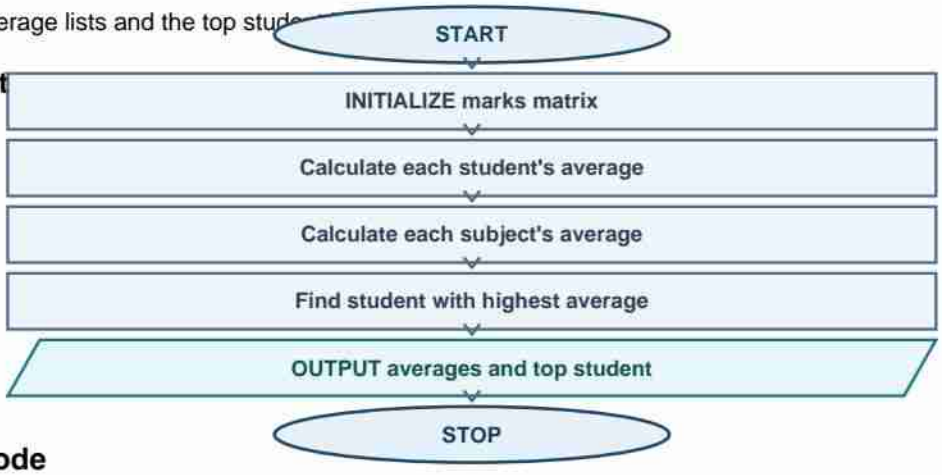
3. For each row, add the marks and divide by the number of subjects. Store the student average.

4. For each column, add the marks of all students and divide by the number of students. Store the subject average.

5. Assume student 0 has the highest average, then compare all student averages and update the index when a higher value is found.

6. Print both average lists and the top stud

2. Flowchart



3. Pseudocode

```
BEGIN
  matrix = [[75,80,85], [60,55,58], [90,92,88], [40,50,45]]
  Find number of students and subjects

  FOR each row in matrix
      total = sum of marks in row
      Add total / number of subjects to student_averages
  END FOR

  FOR each subject column j
      col_total = 0
      FOR each student i
          col_total = col_total + matrix[i][j]
      END FOR
      Add col_total / number of students to subject_averages
  END FOR

  Find the index of the highest student average
  PRINT both average lists and top index
END
```

4. Python Program

```
matrix = [
    [75, 80, 85],
    [60, 55, 58],
    [90, 92, 88],
    [40, 50, 45]
]

num_students = len(matrix)
num_subjects = len(matrix[0])

student_averages = []

for row in matrix:
    total = 0
    for mark in row:
        total = total + mark
    student_averages.append(total / num_subjects)

subject_averages = []

for j in range(num_subjects):
    col_total = 0
    for i in range(num_students):
        col_total = col_total + matrix[i][j]
    subject_averages.append(col_total / num_students)

top_student = 0
highest_avg = student_averages[0]

for i in range(len(student_averages)):
    if student_averages[i] > highest_avg:
        highest_avg = student_averages[i]
        top_student = i

print("Student Averages:", student_averages)
print("Subject Averages:", subject_averages)
print("Top Student Index:", top_student)
```

5. Sample Output

```
Student Averages: [80.0, 57.666666666666664, 90.0, 45.0]
Subject Averages: [66.25, 69.25, 69.0]
Top Student Index: 2
```

6. Test Cases

matrix=[[75,80,85],[60,55,58],[90,92,88],[40,50,45]]	Top Student Index: 2 (average 90.0)
matrix=[[50,50],[80,80]]	Student averages: [50.0, 80.0]; Top student index: 1

Note: The top student index starts from 0 because Python list indexing starts from 0. Therefore index 2 means the third student.

LAB EXPERIMENT – 9

Tuple-Based Product Catalogue Audit

Problem Statement: Store product ID, price, and quantity in tuples, identify invalid records, calculate total inventory value, and list products that need reordering.

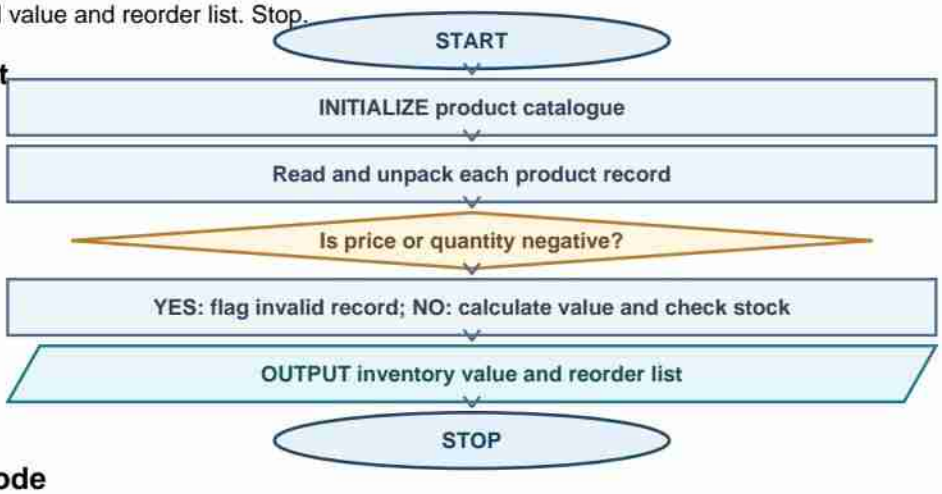
Aim: To unpack tuple records, validate values, and calculate inventory details.

Input – Process – Output: Input: a tuple containing product records in the form (product_id, price, quantity). Process: validate price and quantity, calculate price × quantity, and identify quantities below 5. Output: invalid records, total inventory value, and reorder list.

1. Algorithm

1. Start and initialize the catalogue tuple.
2. Set total_inventory_value = 0 and reorder_list to an empty list.
3. For each product tuple, unpack product ID, price, and quantity.
4. If price or quantity is negative, print the invalid product ID.
5. Otherwise add price × quantity to the inventory value.
6. If quantity is less than 5, add the product ID to reorder_list.
7. Print the total value and reorder list. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  catalogue = (product records)
  total_inventory_value = 0
  reorder_list = []

  FOR each item in catalogue
      Unpack item into pid, price, qty
      IF price < 0 OR qty < 0
          PRINT invalid record and pid
      ELSE
          total_inventory_value = total_inventory_value + price * qty
          IF qty < 5
              APPEND pid to reorder_list
          END IF
      END IF
  END FOR

  PRINT total_inventory_value and reorder_list
END
```

4. Python Program

```
catalogue = (
    ("P101", 150, 10),
    ("P102", -20, 5),
    ("P103", 45, 0),
    ("P104", 80, 2)
)

total_inventory_value = 0
reorder_list = []

for item in catalogue:
    pid, price, qty = item

    if price < 0 or qty < 0:
        print("Invalid item record found:", pid)
    else:
        total_inventory_value = total_inventory_value + (price * qty)

        if qty < 5:
            reorder_list.append(pid)

print("Total Inventory Value:", total_inventory_value)
print("Products needing reorder:", reorder_list)
```

5. Sample Output

```
Invalid item record found: P102
Total Inventory Value: 1660
Products needing reorder: ['P103', 'P104']
```

6. Test Cases

catalogue=(('P101',150,10),('P102',-20,5),('P103',45,0),('P104',80,2))	Invalid: P102; Total value: 1660; Reorder: ['P103', 'P104']
catalogue=(('A1',100,10),('A2',200,1))	Total value: 1200; Reorder: ['A2']

Note: Invalid records are skipped when calculating the inventory value. A quantity of 0 is valid, but it is included in the reorder list because it is below 5.

LAB EXPERIMENT – 10

Bus Passenger Occupancy Simulation

Problem Statement: Track passenger boarding and alighting at multiple bus stops, calculate the running passenger count, and report when the bus exceeds its capacity.

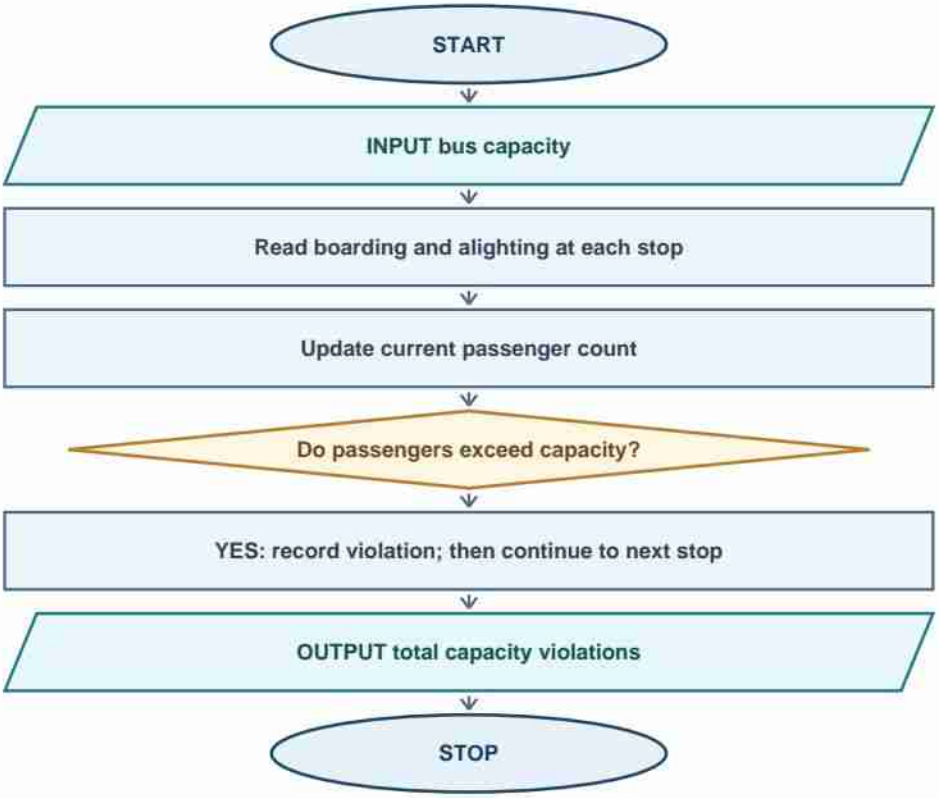
Aim: To update bus occupancy using tuples, a loop, and an if statement.

Input – Process – Output: Input: bus capacity. The sample stop list contains tuples (boarding, alighting). Process: update passenger count at every stop and count capacity violations. Output: passenger count at each stop and total violations.

1. Algorithm

1. Start and read the bus capacity.
2. Set current_passengers = 0, violations = 0, and stop_num = 1.
3. For each stop, unpack boarding and alighting counts.
4. Update current_passengers = current_passengers - alighting + boarding.
5. Print the passenger count.
6. If current_passengers is greater than capacity, print a violation message and increase violations by 1.
7. Increase stop_num by 1 and continue for all stops.
8. Print total violations. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN
  INPUT capacity
  current_passengers = 0
  violations = 0
  stop_num = 1
  stops = [(15,0), (10,5), (25,2), (5,10)]

  FOR each stop in stops
      Unpack stop into board, alight
      current_passengers = current_passengers - alight + board
      PRINT stop_num and current_passengers
      IF current_passengers > capacity
          PRINT capacity violation
          violations = violations + 1
      END IF
      stop_num = stop_num + 1
  END FOR

  PRINT violations
END
```

4. Python Program

```
capacity = int(input("Enter bus capacity: "))

current_passengers = 0
violations = 0
stops = [(15, 0), (10, 5), (25, 2), (5, 10)]
stop_num = 1

for stop in stops:
    board, alight = stop
    current_passengers = current_passengers - alight + board

    print("Stop", stop_num, "- Passengers on board:", current_passengers)

    if current_passengers > capacity:
        print("Capacity violation at stop", stop_num)
        violations = violations + 1

    stop_num = stop_num + 1

print("Total Capacity Violations:", violations)
```

5. Sample Output

```
Enter bus capacity: 40
Stop 1 - Passengers on board: 15
Stop 2 - Passengers on board: 20
Stop 3 - Passengers on board: 43
Capacity violation at stop 3
Stop 4 - Passengers on board: 38
Total Capacity Violations: 1
```

6. Test Cases

capacity=40, stops=[(15,0),(10,5),(25,2),(5,10)]	Capacity violation at stop 3; passenger count at stop 3 is 43; total violations: 1
capacity=50, stops=[(20,0),(10,5)]	Stop 1: 20 passengers; Stop 2: 25 passengers; total violations: 0

Note: The supplied manual's test case says 48 passengers at stop 3, but its listed stop tuples calculate to 43. This PDF uses the stated formula and tuples, so the correct count for those inputs is 43.